Algorithmic Bias Audit Report

Target File: `./test/assign_holiday_schedule_.py`

Audit Model: FATE Auditor Model

Analysis Type: Static Code FATE Analysis

Status: 3 Flags Detected

Executive Summary

This report details instances of algorithmic bias identified within the source code during automated static analysis. The evaluation surfaced exclusionary logic based on protected attributes and/or proxies, which may propagate differential treatment through the Software Development Life Cycle (SDLC).

Detailed Findings

Finding 1: Proxy Bias

Association: NEGATIVE

Non-Protected Attribute: holiday_routine_familiarity

Biased Code Snippet:

```
employee["holiday_routine_familiarity"] >= 0.8
```

Issue Description: The attribute 'holiday_routine_familiarity' could act as a proxy for protected attributes like religion or nationality, as familiarity with certain holidays is often tied to cultural background. Using it to assign holiday work may disproportionately affect certain groups.

Impact Reasoning: Relies on a measurable attribute ('holiday_routine_familiarity') as a stand-in for a trait that correlates with a protected group, unfairly impacting groups where that attribute does not accurately indicate the intended outcome.

Finding 2: Threshold/Cutoff Bias

Association: NEGATIVE

Non-Protected Attribute: holiday_routine_familiarity

Biased Code Snippet:

```
employee["holiday_routine_familiarity"] >= 0.8
```

Issue Description: The code uses a fixed cutoff of 0.8 to decide who gets scheduled for holiday work, creating a hard boundary that may unfairly disadvantage employees who fall just below the threshold.

Impact Reasoning: Uses a fixed non-protected attribute ('holiday_routine_familiarity >= 0.8') with a threshold to determine eligibility or outcomes, where the rigid cutoff may produce biased treatment for individuals near the boundary.

Finding 3: Demographic Bias

Association: NEGATIVE

Non-Protected Attribute: holiday_routine_familiarity

Biased Code Snippet:

```
        if employee["holiday_routine_familiarity"] >= 0.8:
                    scheduled.append(employee)
```

Issue Description: The code creates two distinct groups of employees (scheduled and not scheduled) based on the 'holiday_routine_familiarity' attribute, leading to differential treatment for holiday work assignments.

Impact Reasoning: Uses a non-protected attribute ('holiday_routine_familiarity') to result in differential treatment of individuals based on group membership or proxy grouping.